

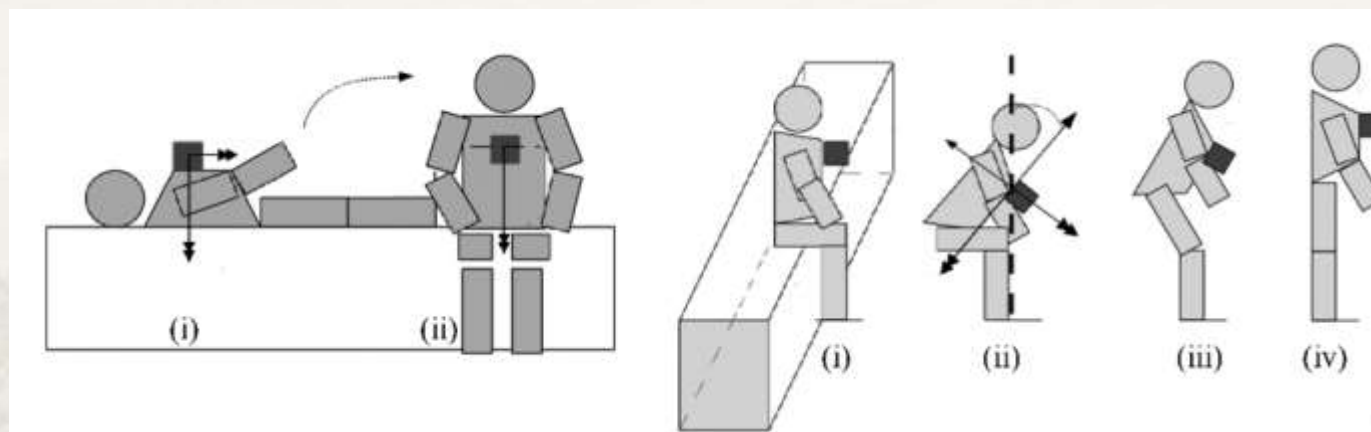
第五讲 基于“集体智慧”的预测算法 ---集成学习

主要内容

- 继续讨论案例六的分类预测问题
 - 组合预测进阶：集成学习
 - 三个臭裨将顶一个诸葛亮：从弱模型到强模型的构建
- 继续讨论案例二的回归预测问题
 - 组合预测进阶：集成学习
 - 强强联合：从Bagging到随机森林—聚焦复杂模型的高方差问题

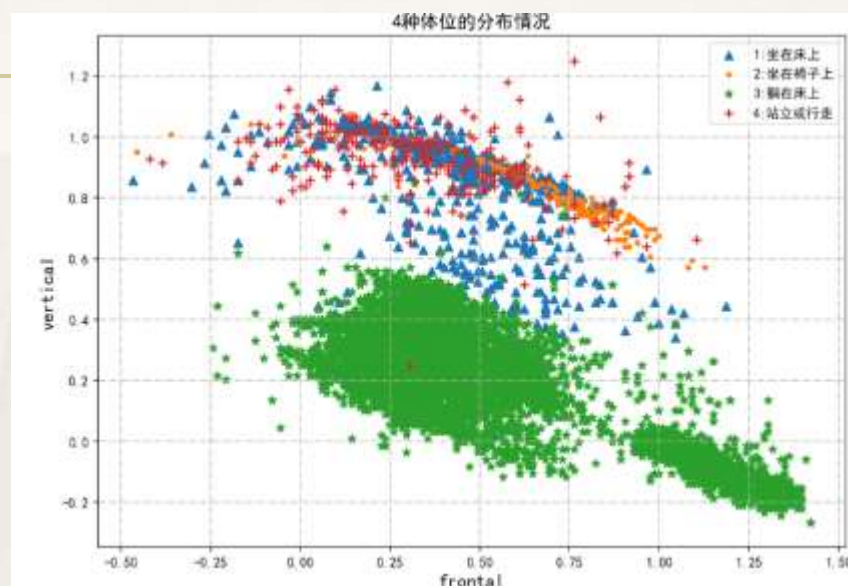
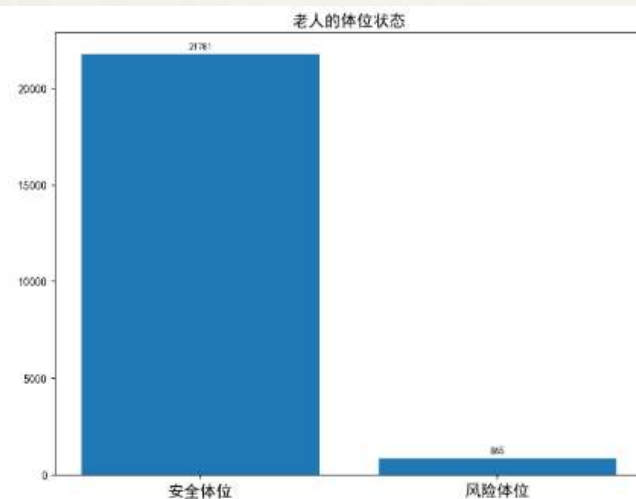
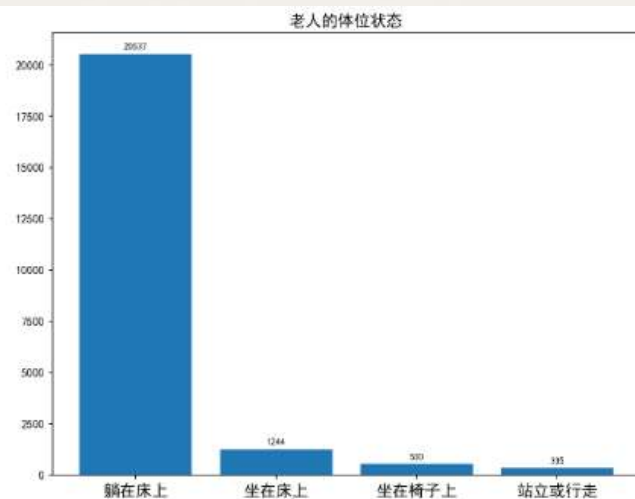
案例六：老年人危险体位预警

• 案例六简介



- 正方性表示无线穿戴设备
- 从平躺在床上，到坐起、起身站起、直立及行走过程中的体位 (Activity) 变化
- 随体位实时变化 (TimeStamp, 不同时间点)，无线穿戴设备的竖直高度 (vertical, 设备距地面的高度)、水平位置 (frontal, lateral, 设备距两个垂直墙体的距离) 数据以及倾角 (Phase) 等都会发生变化
- 数据通过房间内的3个射频识别 (Radio Frequency Identification, RFID) 设备实时采集并传回
 - 其中，2个RFID安装在天花板上，1个安装在墙上
- 老人在室内走动时各设备 (SensorID) 采集的信号强度 (RSSI) 会不同
- 数据采集时段内 (样本量22646)，老人们的体位状态包括
 - 1：坐在床上；2：坐在椅子上；3：躺在床上；4：站立或行走

案例六的SVM多分类预测：完全线性可分→广义线性可分→引入核函数



```
data['ActivityN'] = data['Activity'].map({3:0,1:0,2:1,4:1})
X = data[['frontal', 'vertical', 'lateral', 'RSSI']]
modelSVC = svm.SVC(kernel = 'rbf',C = 50)

y = data['Activity'].astype(int)
modelSVC.fit(X,y)
print('多分类预测情况(训练集):\n',classification_report(y,modelSVC.predict(X)))

y = data['ActivityN'].astype(int)
modelSVC.fit(X,y)
print('二分类预测情况(训练集):\n',classification_report(y,modelSVC.predict(X)))
```

训练集上的表现



多分类预测情况(训练集):

	precision	recall	f1-score	support
1	0.70	0.76	0.73	1244
2	0.81	0.70	0.75	530
3	0.99	1.00	0.99	20537
4	0.98	0.27	0.42	335
accuracy			0.97	22646
macro avg	0.87	0.68	0.72	22646
weighted avg	0.97	0.97	0.97	22646

二分类预测情况(训练集):

	precision	recall	f1-score	support
0	0.97	1.00	0.98	21781
1	0.93	0.16	0.27	865
accuracy			0.97	22646
macro avg	0.95	0.58	0.63	22646
weighted avg	0.97	0.97	0.96	22646

案例六的SVM多分类预测：完全线性可分→广义线性可分→引入核函数

重采样(平衡处理)上的简单组合预测

训练误差 (重采样数据集)				
	precision	recall	f1-score	support
0	1.00	0.95	0.98	19611
1	0.96	1.00	0.98	19587
accuracy			0.98	39198
macro avg	0.98	0.98	0.98	39198
weighted avg	0.98	0.98	0.98	39198

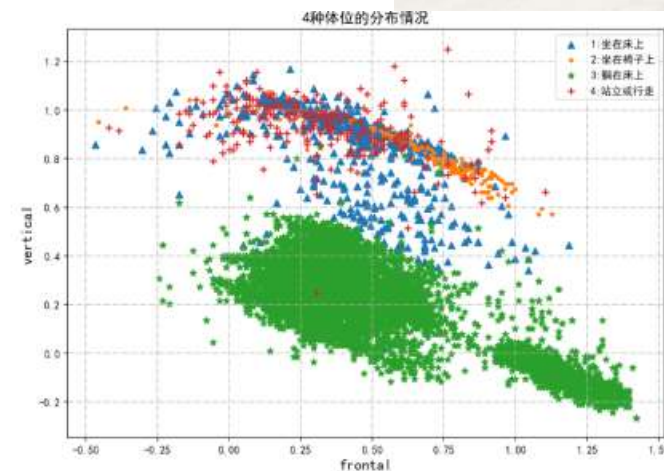
测试误差				
	precision	recall	f1-score	support
0	1.00	0.96	0.98	2170
1	0.51	1.00	0.68	95
accuracy			0.96	2265
macro avg	0.76	0.98	0.83	2265
weighted avg	0.98	0.96	0.97	2265

重采样(平衡处理)上的Stacking策略

训练误差 (重采样数据集)				
	precision	recall	f1-score	support
0	1.00	1.00	1.00	19611
1	1.00	1.00	1.00	19587
accuracy			1.00	39198
macro avg	1.00	1.00	1.00	39198
weighted avg	1.00	1.00	1.00	39198

测试误差				
	precision	recall	f1-score	support
0	1.00	0.97	0.99	2170
1	0.60	0.95	0.74	95
accuracy			0.97	2265
macro avg	0.80	0.96	0.86	2265
weighted avg	0.98	0.97	0.97	2265

- 问：如何在**不改变数据分布**下做得更好些？
 - 可否采用树模型？适用性更广(这里为模型比较未引入新变量)
 - 可否让各基学习器“**聚焦**”不同的**局部数据**(尤其是错判数据)并进行**联合预测**？
 - 联合预测——集体智慧：集成学习



集成学习：从弱模型到强模型(三个臭裨将顶一个诸葛亮)

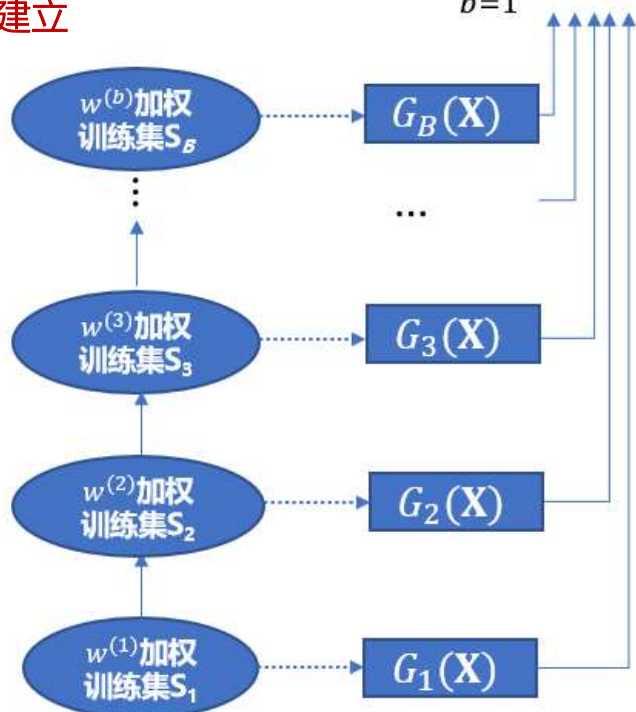
- 将一组弱模型组成一个“联合委员会”并最终成为强模型
 - 弱模型：一般指比随机猜测的误差略低的模型
 - 二分类预测中的随机猜测误差等于0.5
 - 零模型 (Zero Model) 一种典型的弱模型；比零模型略好的弱模型将考虑输入变量
- 通过“联合委员会”的联合预测成为强模型。如回归预测中：
 - B 个弱模型对 X_0 的回归预测值分别为： $\hat{f}^{*(1)}(X_0), \hat{f}^{*(2)}(X_0), \dots, \hat{f}^{*(B)}(X_0)$,
 - 联合预测结果为： $\hat{f}_\alpha(X_0) = \alpha_1 \hat{f}^{*(1)}(X_0) + \alpha_2 \hat{f}^{*(2)}(X_0) + \dots + \alpha_B \hat{f}^{*(B)}(X_0)$
 - $\alpha_b (b = 1, 2, \dots, B)$ 为模型权重； B 个弱模型具有顺序相关性
- 弱模型到强模型的常见集成学习策略
 - 提升 (Boosting)
 - 梯度提升算法和梯度提升树
 - 梯度提升树的XGBoost、LightGBM算法，等

- Boosting: 一类集成学习策略的统称
 - 经典是AdaBoost (Adaptive Boosting) 适应性提升法, 提出时针对输出变量 $y \in \{-1, +1\}$ 的**二分类**预测, 后拓展到多分类场景
- AdaBoost的**基本框架**:

联合委员会对 $\mathbf{x}_i$ 的联合预测:

$$\hat{y}_i = \text{sign}(f_B(\mathbf{x}_i)) = \text{sign}\left(\sum_{b=1}^B \alpha_b G_b(\mathbf{x}_i)\right)$$

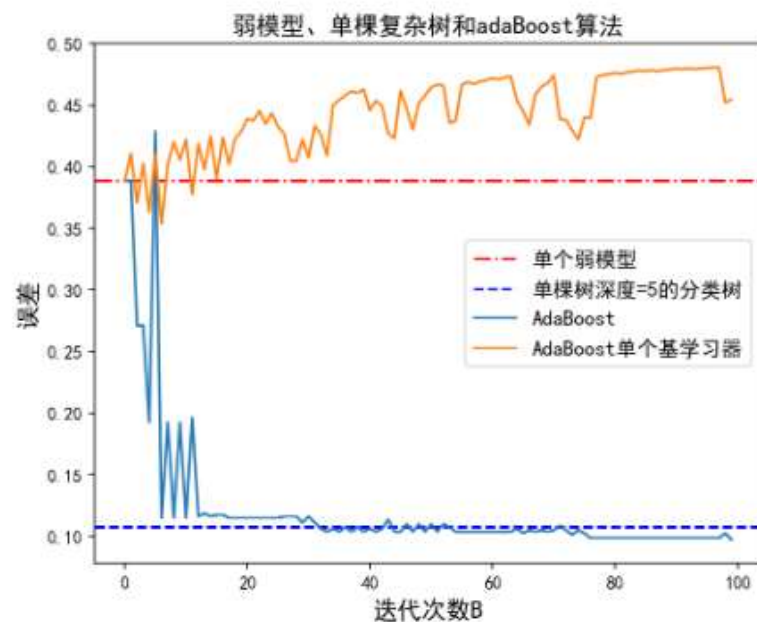
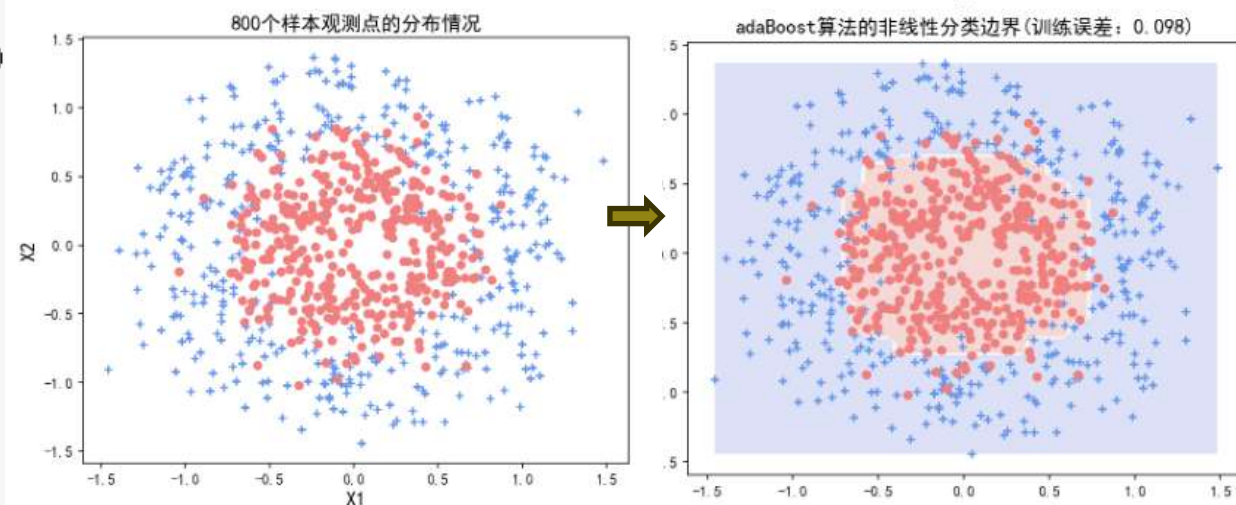
通过 B 次迭代建立
 B 个弱模型



- 每次迭代做**两**件事情:
 - 基于权重 $w^{(b)}$ 下的训练集 S_b 建立弱模型 $G_b(\mathbf{X})$
 - 基于权重 $w^{(b)}$ 得到更新的权重 $w^{(b+1)}$
- 预测值 $G_b(\mathbf{x}_i) \in \{-1, +1\}$, 只需根据 $\sum_{b=1}^B \alpha_b G_b(\mathbf{x}_i)$ 的符号便可给出联合预测结果 $\hat{y}_i$
- AdaBoost 框架训练误差为: $err = \frac{1}{N} \sum_{i=1}^N I(y_i \neq \hat{y}_i)$
 - 测试误差: 基于 OOB 计算的误差
- 问:
 - **弱模型**的联合可以成为**强模型**吗?

• 数据模拟和启示：弱模型的联合可以成为强模型吗？ Chapter8-2.ipynb

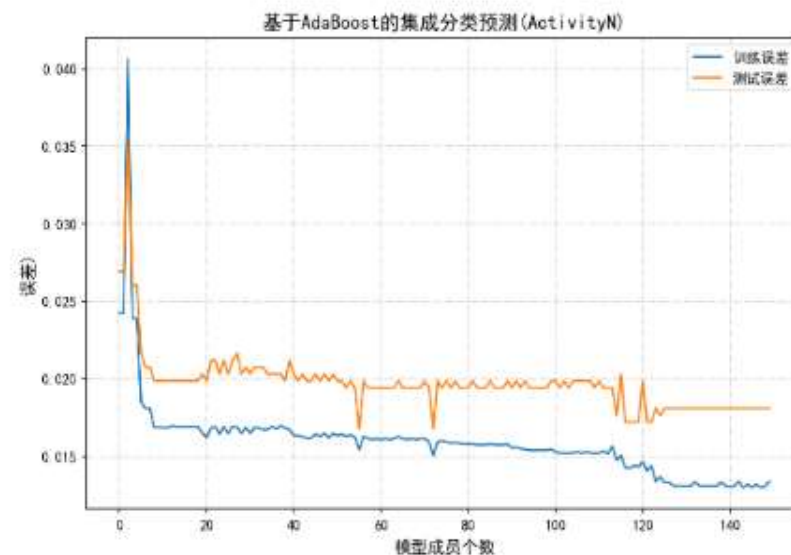
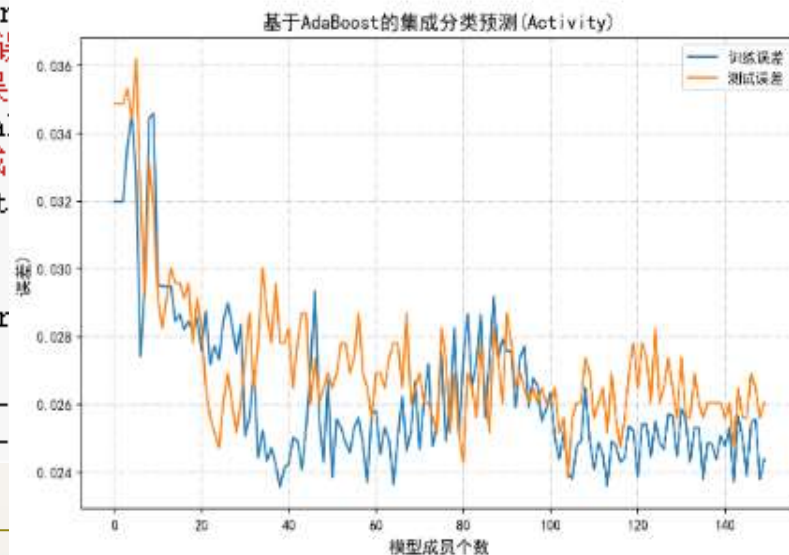
```
1 N = 800
2 X,y = make_circles(n_samples = N,noise = 0.2,factor = 0.5,random_state = 123)
3 fig,axes = plt.subplots(nrows = 2,ncols = 2,figsize = (15,12),dpi = 150)
4 colors = ['cornflowerblue','LightCoral']
5 markers = ['+', 'o']
6 for k,col,m in zip(set(y),colors,markers):
7     axes[0,0].scatter(X[y == k,0],X[y == k,1],color = col,s = 30,marker = m)
8 axes[0,0].set_title('%d个样本观测点的分布情况'%N,fontsize = 14)
9 axes[0,0].set_xlabel('X1',fontsize = 14)
10 axes[0,0].set_ylabel('X2',fontsize = 14)
11 dt_stump = tree.DecisionTreeClassifier(max_depth = 1,random_state = 123)
12 dt_stump.fit(X, y)
13 dt_stump_err = 1.0 - dt_stump.score(X, y)
14 dt = tree.DecisionTreeClassifier(max_depth = 5,random_state = 123)
15 dt.fit(X, y)
16 dt_err = 1.0 - dt.score(X, y)
17 B = 100 #可能会早停
18 ada = ensemble.AdaBoostClassifier(base_estimator = dt_stump,n_estimators = B,random_state = 123,)
19 ada.fit(X, y)
20 ada_err = np.zeros((B,))
21 for i,yhat in enumerate(ada.staged_predict(X)):
22     ada_err[i] = zero_one_loss(yhat, y)
23 axes[0,1].axhline(y = dt_stump_err,c = 'red',linestyle = '-',label = '单个弱模型')
24 axes[0,1].axhline(y = dt_err,c = 'blue',linestyle = '--',label = '单棵树深度=5的分类树')
25 axes[0,1].plot(ada_err,linestyle = '-',label = 'AdaBoost')
26 axes[0,1].plot(ada.estimator_errors,linestyle = '-',label = 'AdaBoost单个基学习器')
27 axes[0,1].set_title('弱模型、单棵复杂树和adaBoost算法',fontsize = 14)
28 axes[0,1].set_xlabel('迭代次数B',fontsize = 14)
29 axes[0,1].set_ylabel('误差',fontsize = 14)
```



弱模型的联合预测有较低的预测误差

• 案例六的分析：弱模型的联合可以成为强模型——基于Boosting策略的预测

```
1 X = data[['frontal', 'vertical', 'lateral', 'RSSI']] #树模型数据可不进行标准化处理
2 modelDTC = tree.DecisionTreeClassifier(max_depth = 2, random_state = 123)
3 fig, axes = plt.subplots(nrows=1, ncols=2, figsize=(20, 6), dpi=150)
4
5 for l, Y in zip([0, 1], ['Activity', 'ActivityN']):
6     y = data[Y].astype(int)
7     X_train, X_test, y_train, y_test = train_test_split(X, y, train_size = 0.90, random_state = 123)
8     B = 150
9     ErrTrainAdaB = np.zeros((B,)); ErrTestAdaB = np.zeros((B,))
10    adaBoost = ensemble.AdaBoostClassifier(base_estimator = modelDTC, n_estimators = B, random_state = 123, algorithm
11    adaBoost.fit(X_train, y_train)
12
13    for b, yhat in enumerate(adaBoost.staged_predict(X_train)):
14        ErrTrainAdaB[b] = zero_one_loss(yhat, y_train)
15    for b, yhat in enumerate(adaBoost.staged_predict(X_test)):
16        ErrTestAdaB[b] = zero_one_loss(yhat, y_test)
17    bestB = np.where(ErrTestAdaB == np.min(ErrTestAdaB))[0][0]
18    axes[l].plot(ErrTrainAdaB, label='训练误差')
19    axes[l].plot(ErrTestAdaB, label='测试误差')
20    axes[l].grid(True, linestyle = '-.', alpha = 0.5)
21    axes[l].set_title(f'基于AdaBoost的集成分类预测({Y})')
22    axes[l].set_xlabel('模型成员个数', fontdict={'size': 12})
23    axes[l].set_ylabel('误差', fontdict={'size': 12})
24    axes[l].legend(fontsize = 10)
25    adaBoost = ensemble.AdaBoostClassifier(base_estimator = modelDTC, n_estimators = B, random_state = 123, algorithm
26    adaBoost.fit(X_train, y_train)
27    print(f'训练集({Y})\n', classification_report(y_train, yhat))
28    print(f'测试集({Y})\n', classification_report(y_test, yhat))
```



• 案例六的分析：弱模型的联合可以成为强模型——基于Boosting策略的预测

多分类预测情况(训练集):

	precision	recall	f1-score	support
SVM直接预测				
1	0.70	0.76	0.73	1244
2	0.81	0.70	0.75	530
3	0.99	1.00	0.99	20537
4	0.98	0.27	0.42	335
accuracy			0.97	22646
macro avg	0.87	0.68	0.72	22646
weighted avg	0.97	0.97	0.97	22646

二分类预测情况(训练集):

	precision	recall	f1-score	support
SVM直接预测				
0	0.97	1.00	0.98	21781
1	0.93	0.16	0.27	865
accuracy			0.97	22646
macro avg	0.95	0.58	0.63	22646
weighted avg	0.97	0.97	0.96	22646

重采样(平衡处理)上的Stacking策略

测试误差

	precision	recall	f1-score	support
SVM预测				
0	1.00	0.97	0.99	2170
1	0.80	0.95	0.74	95
accuracy			0.97	2265
macro avg	0.80	0.96	0.86	2265
weighted avg	0.98	0.97	0.97	2265

训练集(Activity)

	precision	recall	f1-score	support
Boosting				
1	0.78	0.81	0.79	1133
2	0.88	0.72	0.79	474
3	1.00	1.00	1.00	18478
4	0.65	0.57	0.61	296
accuracy			0.98	20381
macro avg	0.82	0.77	0.80	20381
weighted avg	0.98	0.98	0.98	20381

测试集(Activity)

	precision	recall	f1-score	support
1	0.75	0.77	0.76	111
2	0.92	0.79	0.85	56
3	0.99	1.00	1.00	2059
4	0.68	0.59	0.63	39
accuracy			0.98	2265
macro avg	0.84	0.79	0.81	2265
weighted avg	0.98	0.98	0.98	2265

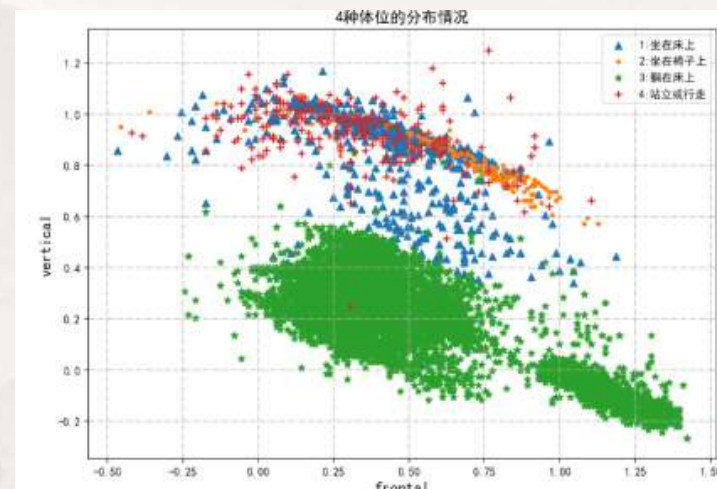
训练集(ActivityN)

	precision	recall	f1-score	support
Boosting				
0	0.99	0.99	0.99	19611
1	0.84	0.72	0.78	770
accuracy			0.98	20381
macro avg	0.92	0.86	0.89	20381
weighted avg	0.98	0.98	0.98	20381

测试集(ActivityN)

	precision	recall	f1-score	support
0	0.99	0.99	0.99	2170
1	0.85	0.73	0.78	95
accuracy			0.98	2265
macro avg	0.92	0.86	0.89	2265
weighted avg	0.98	0.98	0.98	2265

- 问：Boosting理想的原因？
 - 让各基学习器向不尽相同的训练集学习



- 通过给样本观测以合理的不尽相同的权重实现
- 问：
 - 如何赋权？
 - 为什么这样赋权？

• AdaBoost.M1算法：AdaBoost中应用最为广泛的算法,关注两个权重：

• **样本观测权重**： $\mathbf{w}^{(b)} = (w_1^{(b)}, w_2^{(b)}, \dots, w_N^{(b)})$, **模型权重**： α_b

• 第一次迭代：权重 $\mathbf{w}^{(1)} = (w_1^{(1)}, w_2^{(1)}, \dots, w_N^{(1)})$ 等于初始值 $\frac{1}{N}$ 。基于权重 $\mathbf{w}^{(1)}$ 下的训练集 S_1 是原始训练集 S ，建立弱模型 $G_1(X)$

• 基于 $\mathbf{w}^{(1)}$ 得到更新的 $\mathbf{w}^{(2)}$

• 计算弱模型 $G_1(X)$ 的训练误差：

$$err^{(1)} = \frac{1}{N} \sum_{i=1}^N I(y_i \neq G_1(X_i)) = \frac{\sum_{i=1}^N w_i^{(1)} I(y_i \neq G_1(X_i))}{\sum_{i=1}^N w_i^{(1)}} < 0.5 \quad \text{为什么?}$$

• 依据训练误差 $err^{(1)}$ 设置联合预测中 $G_1(X)$ 的模型权重：

$$\alpha_1 = \log\left(\frac{1-err^{(1)}}{err^{(1)}}\right) > 0$$

• 基于 α_1 和 $\mathbf{w}^{(1)}$ 得到更新的 $\mathbf{w}^{(2)}$ ：

$$w_i^{(2)} = w_i^{(1)} \exp\left(\alpha_1 I(y_i \neq G_1(X_i))\right), i = 1, 2, \dots, N$$

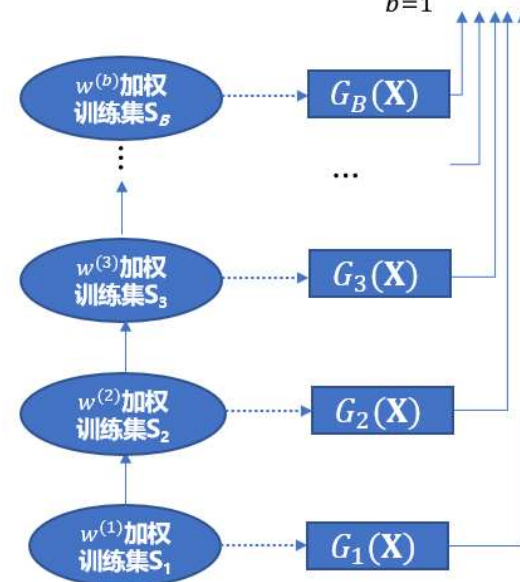
• 若 $G_1(X)$ 对 X_i 预测**正确**： $w_i^{(2)} = w_i^{(1)}$ ，**权重不变**

• 若预测**错误**： $w_i^{(2)} = w_i^{(1)} \times \frac{1-err^{(1)}}{err^{(1)}}$ ， $w_i^{(2)} > w_i^{(1)}$ ，**权重变大**

• 错误预测下权重 w_i ，是若被正确预测下的 $\frac{1-err^{(1)}}{err^{(1)}}$ 倍

联合委员会对 X_i 的联合预测：

$$\hat{y}_i = \text{sign}(f_B(X_i)) = \text{sign}\left(\sum_{b=1}^B \alpha_b G_b(X_i)\right)$$



- AdaBoost.M1算法：AdaBoost中应用最为广泛的算法，关注两个权重：
 - **样本观测权重**： $\mathbf{w}^{(b)} = (w_1^{(b)}, w_2^{(b)}, \dots, w_N^{(b)})$ ，**模型权重**： α_b
 - 第一次迭代：权重 $\mathbf{w}^{(1)} = (w_1^{(1)}, w_2^{(1)}, \dots, w_N^{(1)})$ 等于初始值 $\frac{1}{N}$ 。基于权重 $\mathbf{w}^{(1)}$ 下的训练集 $\mathcal{S}_1$ 是原始训练集 $\mathcal{S}$ ，建立弱模型 $G_1(X)$
 - 第二次迭代：
 - 依 $\mathbf{w}^{(2)}$ 对 $\mathcal{S}$ 进行有放回的随机抽样得到 $\mathcal{S}_2$ (样本量等于 N)
 - $G_1(X)$ 预测错误的样本观测有更大的概率进入训练集 $\mathcal{S}_2$
 - 基于 $\mathcal{S}_2$ 建立 $G_2(X)$
 - 基学习器 $G_2(X)$ 应至少是个弱模型！
 - 模型 $G_2(X)$ 关注的是 $G_1(X)$ 预测错误的样本观测，两者存在前后顺序关系
 - 计算模型 $G_2(X)$ 的训练误差 $err^{(2)}$ 并依此计算权重 α_2 ，基于 $\mathbf{w}^{(2)}$ 得到更新的 $\mathbf{w}^{(3)}$
 - 第三次至第 B 次迭代同理

- AdaBoost.M1算法：AdaBoost中应用最为广泛的算法，关注两个权重：

- 样本观测权重： $\mathbf{w}^{(b)} = (w_1^{(b)}, w_2^{(b)}, \dots, w_N^{(b)})$ ，模型权重： α_b

- $G_b(\mathbf{X})$ 专注 $G_{b-1}(\mathbf{X})$ 预测错误的观测， $G_b(\mathbf{X})$ 和 $G_{b-1}(\mathbf{X})$ 存在前后顺序关系

$$err^{(b)} = \frac{\sum_{i=1}^N w_i^{(b)} I(y_i \neq G_b(\mathbf{X}_i))}{\sum_{i=1}^N w_i^{(b)}}$$

$$\alpha_b = \log \frac{1 - err^{(b)}}{err^{(b)}}$$

$$\alpha_b = \log \frac{1 - err^{(b)}}{err^{(b)}} + \log(K - 1)$$

$$w_i^{(b+1)} = w_i^{(b)} \exp(\alpha_b I(y_i \neq G_b(\mathbf{X}_i))), i = 1, 2, \dots, N$$

$K > 2$ 分类预测算法称为：

SAMME(Stagewise Additive Modeling using a Multi-class Exponential loss function)

- 被 $G_b(\mathbf{X})$ 错误预测的样本观测 $\mathbf{X}_i$ 权重是被正确预测下的 $\exp(\alpha_b I(y_i \neq G_b(\mathbf{X}_i))) = \frac{1 - err^{(b)}}{err^{(b)}}$ 倍
 - 错误预测的样本观测在 $b + 1$ 次迭代时将有更大的概率进入训练集 $\mathbf{S}_{b+1}$
- 问：
 - 权重较大的样本观测一般有什么特点？

• 数据模拟与启示：图形化展示不同迭代次数下各样本观测的权重变化

Chapter8-3.ipynb

权重较大的点是之前弱模型没有正确预测的点，主要集中在两类的边界处

```

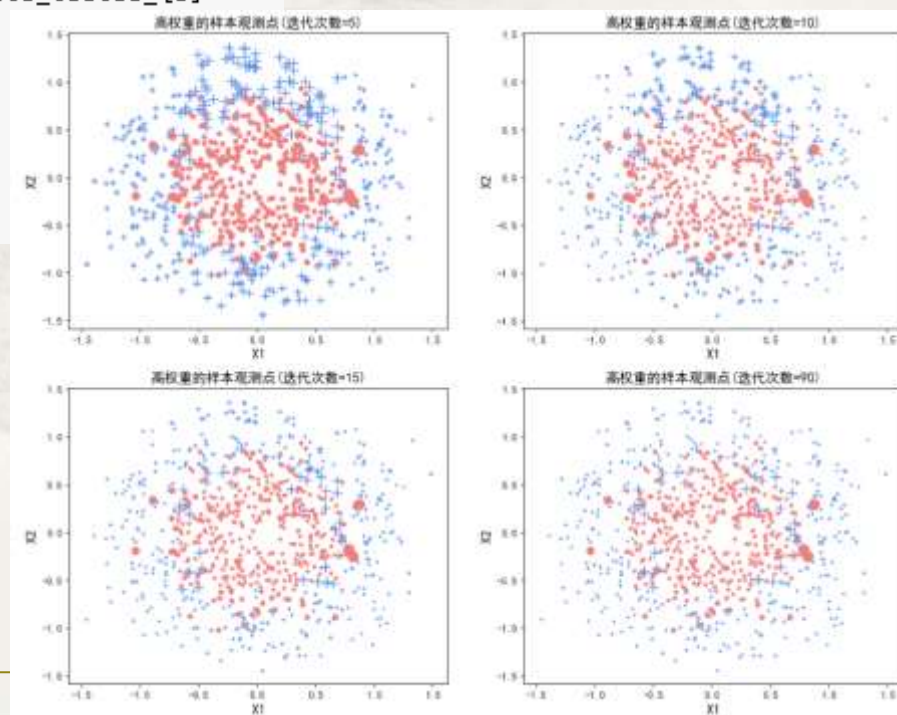
1 N = 800
2 X,y = make_circles(n_samples = N,noise = 0.2,factor = 0.5,random_state = 123)
3 data = np.hstack((X.reshape(N,2),y.reshape(N,1)))
4 data = pd.DataFrame(data);data.columns = ['X1','X2','y']
5 data['Weight'] = [1/N]*N
6 dt_stump = tree.DecisionTreeClassifier(max_depth = 1, min_samples_leaf = 1)
7 B = 100
8 ada = ensemble.AdaBoostClassifier(base_estimator = dt_stump,n_estimators = B,algorithm='SAMME',random_state=123)
9 ada.fit(X, y)
10 fig = plt.figure(figsize = (15,12),dpi = 150)
11 colors = ['cornflowerblue','LightCoral']
12 markers = ['+', 'o']
13 for b,yhat in enumerate(ada.staged_predict(X)):
14     data['yhat'] = yhat
15     data.loc[data['y'] != data['yhat'],'Weight'] *= (1.0-ada.estimator_errors_[b])/ada.estimator_errors_[b]
16     if b in [4,9,14,89]:
17         axes = fig.add_subplot(2,2,[4,9,14,89].index(b)+1)
18         for k,col,m in zip(set(y),colors,markers):
19             tmp =data.loc[data['y'] == k,:]
20             tmp['Weight'] = 10+tmp['Weight']/(tmp['Weight'].max()-tmp['Weight'].min())*100
21             axes.scatter(tmp['X1'],tmp['X2'],color = col,s = tmp['Weight'],marker = m)
22             axes.set_title('高权重的样本观测点(迭代次数=%d)'%(b+1),fontsize = 14)

```

$$w_i^{(b+1)} = w_i^{(b)} \times \frac{1 - err^{(b)}}{err^{(b)}}$$

• 问：

- 对错误预测的样本为什么这样更新权重？
- 算法的损失函数是什么？是如何参与建模的？



- 问：对错误预测的样本为什么这样更新权重？损失函数是什么？如何参与建模？

- AdaBoost.M1算法的深入理解： $\mathbf{X}_i$ 的预测结果取决于 $f_B(\mathbf{X}_i) = \sum_{b=1}^B \alpha_b G_b(\mathbf{X}_i)$ 的正负

$$f_B(\mathbf{X}_i) = \sum_{b=1}^B \alpha_b G_b(\mathbf{X}_i) = \sum_{b=1}^{B-1} \alpha_b G_b(\mathbf{X}_i) + \alpha_B G_B(\mathbf{X}_i) = f_{B-1}(\mathbf{X}_i) + \alpha_B G_B(\mathbf{X}_i)$$

$$f_b(\mathbf{X}_i) = f_{b-1}(\mathbf{X}_i) + \alpha_b G_b(\mathbf{X}_i)$$

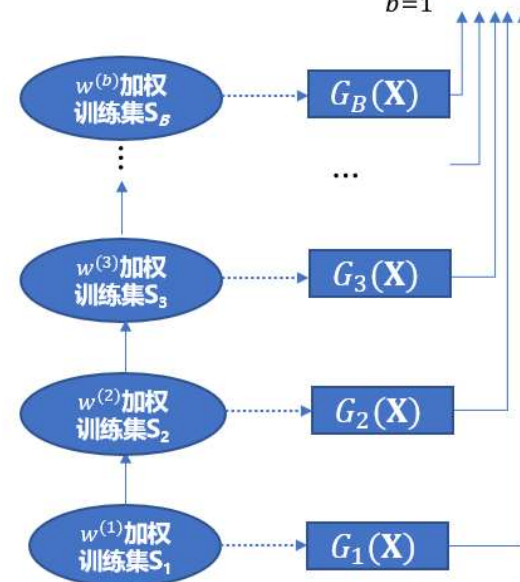
- 迭代次数每增加1次就有一个新的弱模型 $G_b(\mathbf{X}_i)$ 添加 (以加权 α_b 形式) 到当前的“联合委员会”中并参与联合预测

- 向前式分步可加建模 (Forward Stagewise Additive Modeling), 特点:

- 迭代过程是模型成员 $G_b(\mathbf{X})$ 不断进入“联合委员会”的过程
- 先前进入“联合委员会”的模型参数和模型权重, 不受后续进入模型的影响
- 每次迭代仅需估计当前模型 G_b 的参数和权重 α_b

联合委员会对 $\mathbf{X}_i$ 的联合预测:

$$\hat{y}_i = \text{sign}(f_B(\mathbf{X}_i)) = \text{sign}\left(\sum_{b=1}^B \alpha_b G_b(\mathbf{X}_i)\right)$$



- 问：对错误预测的样本为什么这样更新权重？损失函数是什么？如何参与建模？
- 向前式分步可加建模：每次迭代仅需估计当前模型 G_b 的参数和权重 α_b

$$f_b(\mathbf{X}_i) = f_{b-1}(\mathbf{X}_i) + \alpha_b G_b(\mathbf{X}_i)$$

- AdaBoost.M1算法体现了向前式分步可加建模的思想
 - G_b 为决策树时的参数集合记为 γ_b (包括当前最佳分组变量和组限等)，有：

$$f_b(\mathbf{X}) = f_{b-1}(\mathbf{X}) + \beta_b G_b(\mathbf{X}; \gamma_b) \quad \beta_b \text{ 为模型系数 (模型权重 } \alpha_b \text{ 的函数)}$$

- b 次迭代求解损失函数最小时当前模型 G_b 的 γ_b 和模型系数 β_b
 - 第 b 次迭代的总损失函数： $\sum_{i=1}^N L[y_i, f_{b-1}(\mathbf{X}_i) + \beta_b G_b(\mathbf{X}_i; \gamma_b)]$

$$(\beta_b, \gamma_b) = \arg \min_{\beta, \gamma} \sum_{i=1}^N L[y_i, f_{b-1}(\mathbf{X}_i) + \beta G_b(\mathbf{X}_i; \gamma)]$$

- 问：对错误预测的样本为什么这样更新权重？损失函数是什么？如何参与建模？
- AdaBoost.M1算法的深入理解：每次迭代仅需估计当前模型 G_b 的 γ_b 和模型系数 β_b

$$f_b(X) = f_{b-1}(X) + \beta_b G_b(X; \gamma_b) \quad \beta_b \text{ 为模型系数 (权重 } \alpha_b \text{ 的函数)}$$

- 第 b 次迭代时的次迭代时的总损失函数： $\sum_{i=1}^N L[y_i, f_{b-1}(X_i) + \beta_b G_b(X_i; \gamma_b)]$

$$(\beta_b, \gamma_b) = \arg \min_{\beta, \gamma} \sum_{i=1}^N L[y_i, f_{b-1}(X_i) + \beta G_b(X_i; \gamma)]$$

- 采用指数损失函数 ($y \in \{-1, +1\}$) :

- $L(y, f(X)) = \exp(-yf(X))$

$$(\beta_b, \gamma_b) = \arg \min_{\beta, \gamma} \sum_{i=1}^N \exp[-y_i(f_{b-1}(X_i) + \beta G_b(X_i; \gamma))]$$

$$= \arg \min_{\beta, \gamma} \sum_{i=1}^N w_i^{(b)} \exp[-\beta y_i G_b(X_i; \gamma)]$$

$w_i^{(b)} = \exp[-y_i f_{b-1}(X_i)]$ ，是 $b-1$ 迭代完成时 X_i 的损失，作为第 b 次迭代时 X_i 的权重（决定 X_i 进入数据集 S_b 的概率）

- AdaBoost. M1算法的深入理解：因 $G_b(\mathbf{X}_i; \boldsymbol{\gamma}) \in \{-1, +1\}$ ，有：

$$\sum_{i=1}^N w_i^{(b)} \exp[-\beta y_i G_b(\mathbf{X}_i; \boldsymbol{\gamma})] = e^{-\beta} \sum_{y_i = G_b(\mathbf{X}_i)} w_i^{(b)} + e^{\beta} \sum_{y_i \neq G_b(\mathbf{X}_i)} w_i^{(b)} \quad \rightarrow \quad (e^{\beta} - e^{-\beta}) \sum_{i=1}^N w_i^{(b)} I(y_i \neq G_b(\mathbf{X}_i)) + e^{-\beta} \sum_{i=1}^N w_i^{(b)}$$

- 求导并令导数等于0，有： $\beta_b = \frac{1}{2} \log\left(\frac{1 - \text{err}^{(b)}}{\text{err}^{(b)}}\right)$ ，其中 $\text{err}^{(b)} = \frac{\sum_{i=1}^N w_i^{(b)} I(y_i \neq G_b(\mathbf{X}_i))}{\sum_{i=1}^N w_i^{(b)}}$
- 有： $f_b(\mathbf{X}) = f_{b-1}(\mathbf{X}) + \beta_b G_b(\mathbf{X}; \boldsymbol{\gamma}_b)$ ，并得到 $b + 1$ 次迭代的权重为：
 - b 次迭代结束时的损失： $w_i^{(b+1)} = w_i^{(b)} \exp[-\beta_b y_i G_b(\mathbf{X}_i; \boldsymbol{\gamma}_b)]$
 - 因 $-y_i G_b(\mathbf{X}_i) = 2 \cdot I(y_i \neq G_b(\mathbf{X}_i)) - 1$ ，有：
 - $w_i^{(b+1)} = w_i^{(b)} \exp[2\beta_b I(y_i \neq G_b(\mathbf{X}_i))] \exp(-\beta_b)$
 - 等价于： $w_i^{(b+1)} = w_i^{(b)} \exp[\alpha_b I(y_i \neq G_b(\mathbf{X}_i))]$
 - $\alpha_b = 2\beta_b$ 即为**模型权重** $\alpha_b = \log\left(\frac{1 - \text{err}^{(b)}}{\text{err}^{(b)}}\right)$
 - 第 b 次迭代预测错误的样本观测， $b + 1$ 迭代的权重为： $w_i^{(b+1)} = w_i^{(b)} \left(\frac{1 - \text{err}^{(b)}}{\text{err}^{(b)}} \right)$

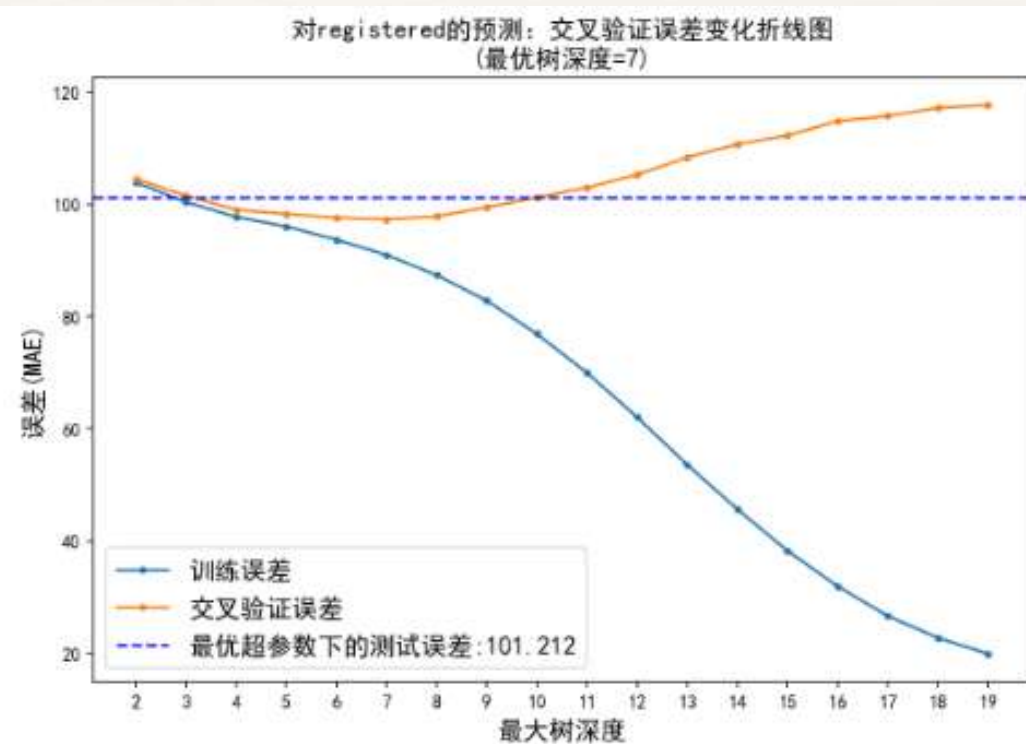
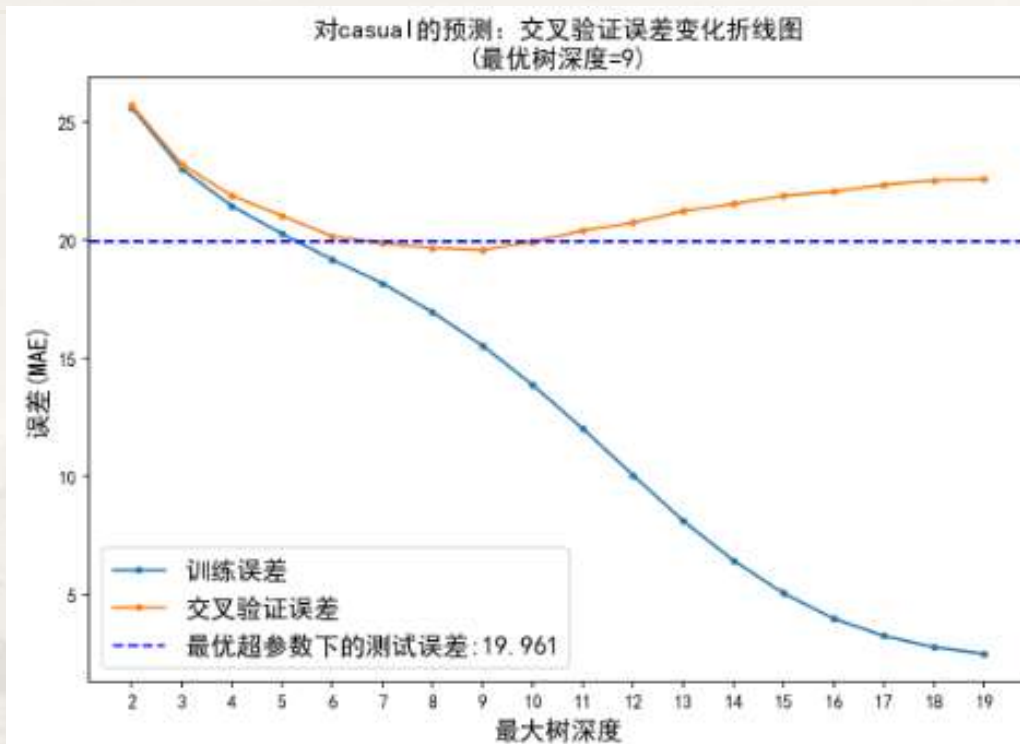
案例二：共享单车日租赁量的预测：回归预测

- 案例二：共享单车日租赁量 (Casual/Registered/count) 的预测：回归预测

	season	holiday	workingday	weather	temp	atemp	humidity	windspeed	casual	registered	count
0	1	0	0	1	9.84	14.395	81	0.0	3	13	16
1	1	0	0	1	9.02	13.635	80	0.0	8	32	40
2	1	0	0	1	9.02	13.635	80	0.0	5	27	32
3	1	0	0	1	9.84	14.395	75	0.0	3	10	13
4	1	0	0	1	9.84	14.395	75	0.0	0	1	1

- Season – 1:spring 2:summer 3:fall 4:winter
- Holiday – whether the day is considered a holiday
- Workingday: whether the day is neither a weekend nor holiday
- Weather -
 - 1: Clear, Few clouds, Partly cloudy
 - 2: Mist + Cloudy, Mist + Broken clouds, Mist + Few clouds, Mist
 - 3: Light Snow, Light Rain + Thunderstorm + Scattered clouds, Light Rain + Scattered clouds
 - 4: Heavy Rain + Ice Pallets + Thunderstorm + Mist, Snow + Fog
- Temp - temperature in Celsius
- Atemp - "feels like" temperature in Celsius
- Humidity - relative humidity
- Windspeed - wind speed
- Casual – number of non-registered user rentals initiated
- Registered - number of registered user rentals initiated
- count - number of total rentals

- 案例二的分析：线性回归分析→K-近邻法→决策树
 - 决策树的预测结果

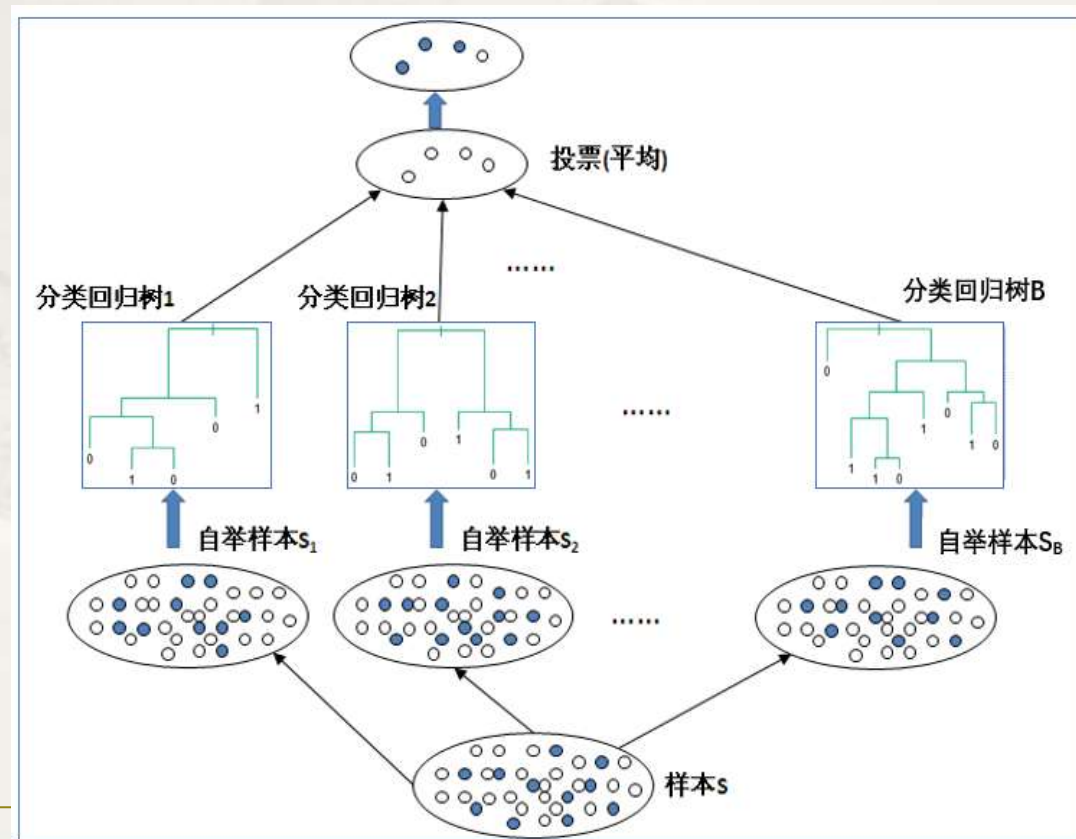


- 问：
 - 可否预测得更好？
 - 基于树模型的“强强联合”：集成学习

集成学习：强强联合的集体智慧

- “强强联合”的基本思路：
 - **建模**阶段：基于 B 组训练集 $S_b (b = 1, 2, \dots, B)$ ，分别建立与之对应的 B 个回归或分类预测模型，称为**基学习器**
 - 基学习器通常为**树深度较大**的决策树，也可以是**其他**复杂预测模型
 - **集成预测**阶段： B 个基学习器给出各自的预测结果 $\hat{f}^{(b)}(X_0)$ ，**集成**预测结果：
 - **回归**： $\hat{f}_{avg}(X_0) = \frac{1}{B} \sum_{b=1}^B \hat{f}^{(b)}(X_0)$
 - **分类**：众数
- **问**：
 - 如何获得 **B 组**训练集 $S_b (b = 1, 2, \dots, B)$ ？
 - “强强联合”可能存在的**问题**？
 - 复杂(强)基学习器的预测方差**怎样**？通过**集成**解决**高方差问题**
 - 通过对多个模型模型预测结果的集成(如平均)，降低预测误差

- 问：如何获得 **B** 组训练集 $S_b (b = 1, 2, \dots, B)$ ？
 - 利用Bootstrap “造” B 组训练集，记为 $S_b^* (b = 1, 2, \dots, B)$
 - Bootstrap是**如何实现的**？ S_b^* 称作什么？
 - 统计基于Bootstrap可以**实现什么**？
- 图中基于 $S_b^* (b = 1, 2, \dots, B)$ 的集成学习策略称为**Bagging** (Bootstrap Aggregating, Bagging) 袋装
 - 特别适合基学习器具有**高方差**和**低偏差**情况
- 问：
 - Bagging中应**如何估计泛化误差**？
 - 计算基于袋外观测 (OOB) 的误差
 - Bagging中**如何判断变量的重要性**？
 - 图中 B 棵树“外观”不同，有特殊**意义**吗？
 - **为什么**不同？



• 问：B棵树“外观”不同，有特殊意义吗？

• 为什么不同？

• 集成学习可以降低预测方差，希望：基于 S_b^* ($b = 1, 2, \dots$)训练的基学习器的预测值 $\hat{f}^{*(1)}(X_0), \hat{f}^{*(2)}(X_0), \dots$ 的方差等于 σ^2 ， $\hat{f}_{avg}^*(X_0) = \frac{1}{B} \sum_{b=1}^B \hat{f}^{*(b)}(X_0)$ 的方差可降至 σ^2/B

• 为什么可行？有前提条件吗？

• 如果前提条件无法满足会怎样？

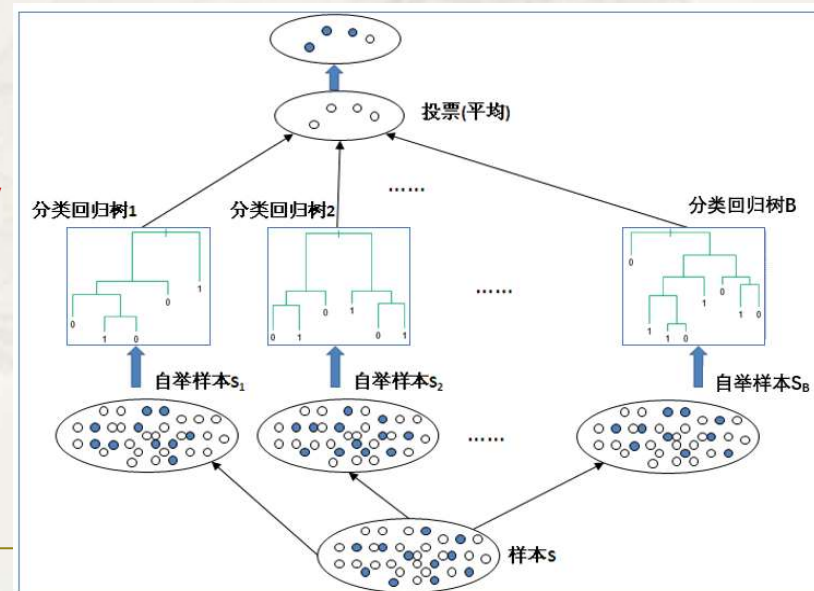
• 假设预测值两两相关系数均等于 ρ ，方差等于： $\rho\sigma^2 + \frac{1-\rho}{B}\sigma^2$

$$\begin{aligned} \text{Var}(\bar{Z}) &= \text{Var}\left(\frac{1}{B} \sum_{i=1}^B Z_i\right) = \frac{1}{B^2} [B\sigma^2 + B(B-1)\rho\sigma^2] \\ &= \frac{\sigma^2 + (B-1)\rho\sigma^2}{B} = \rho\sigma^2 + \frac{1-\rho}{B}\sigma^2 \end{aligned}$$

随B增加第二项趋于0，主要取决第一项

• 你认为Bagging易达成目标吗？

• 可通过哪些策略做出改进？



- 降低树间的相似性减少预测值的**相关性**进而降低方差的更有效策略：**随机森林**
 - **随机森林**用“**随机方式**”建立**多棵决策树**
 - 每棵树都是一个基学习器，“整片”森林对应集成学习
- 随机森林的“**随机方式**”体现在：
 - 对训练数据增加随机性扰动：Bootstrap
 - 对输入变量增加随机性扰动：当前“最佳”分组变量，是来自输入变量的一个随机子集 Θ_b 中的变量
 - **具体过程**：进行 $b = 1, 2, \dots, B$ 次如下迭代
 - 基于自举样本 S_b^* 建立树 $\hat{T}^{*(b)}$ ：树从根节点开始按如下方式不断生长，直到满足树的预修剪参数为止
 - 从 p 个输入变量中随机选择 m 个输入变量构成输入变量的一个随机子集 Θ_b
 - $m = \lfloor \sqrt{p} \rfloor$ 或者 $m = \lfloor \log_2 p \rfloor$
 - 从 Θ_b 中确定当前“最佳”分组变量和分组
 - B 次迭代结束后，会得到包括 B 棵树 $\{\hat{T}^{*(b)}\}^B$ 的随机森林

• 案例二的分析：线性回归分析→K-近邻法→决策树→Bagging/随机森林

```
X = data[['season', 'holiday', 'workingday', 'weather', 'temp', 'atemp', 'humidity', 'windspeed']]
y = data['casual']
X_train, X_test, y_train, y_test = train_test_split(X, y, train_size = 0.90, random_state = 123)
```

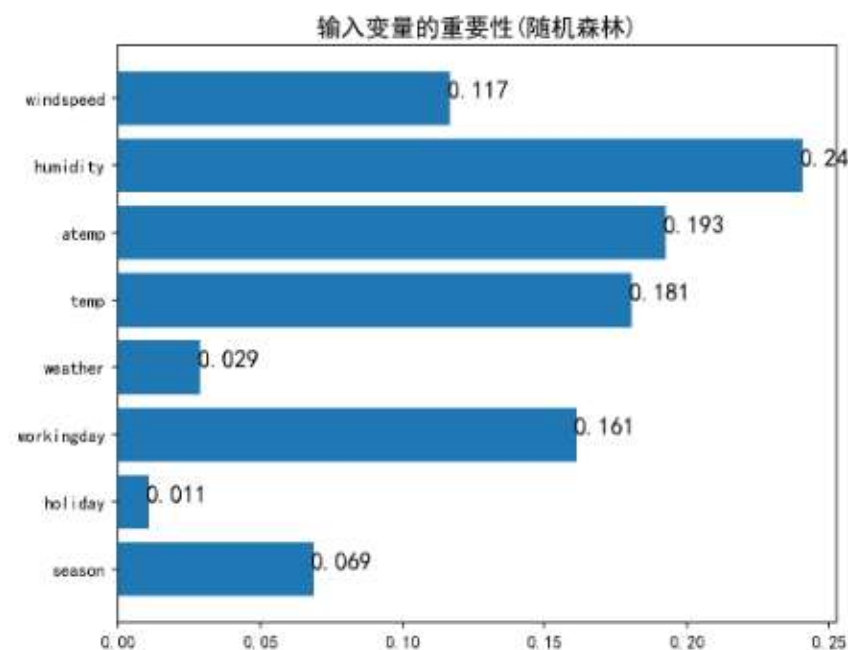
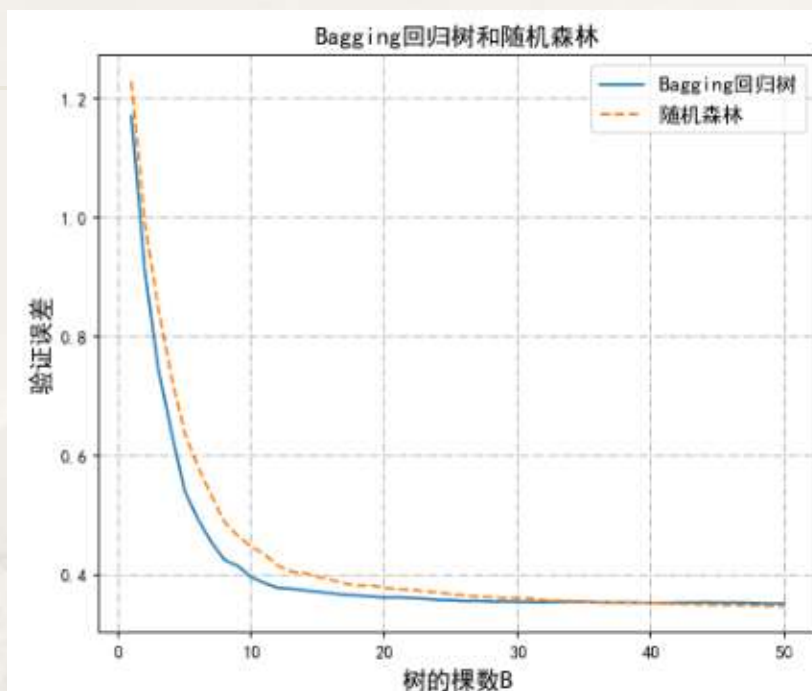
```
bestDeep = 9
modelDTC = tree.DecisionTreeRegressor(max_depth = bestDeep, random_state = 123)
modelDTC.fit(X_train, y_train)
bagValErr = []; rfValErr = [];
B = np.arange(1, 51)
for b in B:
    Bag = ensemble.BaggingRegressor(base_estimator = modelDTC, n_estimators = b,
                                    oob_score = True, random_state = 123, bootstrap = True)
    Bag.fit(X_train, y_train)
    bagValErr.append(1 - Bag.oob_score_)
    RF = ensemble.RandomForestRegressor(n_estimators = b, oob_score = True, random_state = 123,
                                        bootstrap = True, max_features = 'sqrt')
    RF.fit(X_train, y_train)
    rfValErr.append(1 - RF.oob_score_)
```

- 随机森林中变量子集： $m = \lfloor \sqrt{p} \rfloor$
- 随机森林中单棵树为复杂的树(所有叶子“纯”或叶节点样本量不少于2，默认)

```
1 fig, axes = plt.subplots(nrows = 1, ncols = 2, figsize = (16, 6), dpi=150)
2 axes[0].grid(True, linestyle = '-.')
3 axes[0].plot(bagValErr, linestyle = '-.', label = 'Bagging回归树')
4 axes[0].plot(rfValErr, linestyle = '--', label = '随机森林')
5 axes[0].set_title('Bagging回归树和随机森林', fontsize = 14)
6 axes[0].set_xlabel('树的棵数B', fontsize = 14)
7 axes[0].set_ylabel('验证误差', fontsize = 14)
8 axes[0].legend(fontsize = 12)
9 axes[1].barh(y = np.arange(1, X.shape[1]+1), width = RF.feature_importances_, tick_label = X.columns)
10 axes[1].set_title('输入变量的重要性(随机森林)', fontsize = 14)
11 for w1, w2 in enumerate(RF.feature_importances_):
12     axes[1].text(w2 + 0.01, w1 + 1, '%s' % round(w2, 3), ha = 'center', fontsize = 14)
```

- 案例二的分析：线性回归分析→K-近邻法→决策树→Bagging/随机森林

可利用验证误差
进行超参数(如B,
最大树深度等)调优



```
dtrTestErr = mean_absolute_error(y_test,modelDTC.predict(X_test))
bagTestErr = mean_absolute_error(y_test,Bag.predict(X_test))
rfTestErr = mean_absolute_error(y_test,RF.predict(X_test))
print(f'测试误差(MAE): \n    单课决策树={dtrTestErr:.3f}\n    Bagging={bagTestErr:.3f}\n    随机森林={rfTestErr:.3f}')
```

测试误差(MAE):
单课决策树=19.961
Bagging=18.928
随机森林=18.011

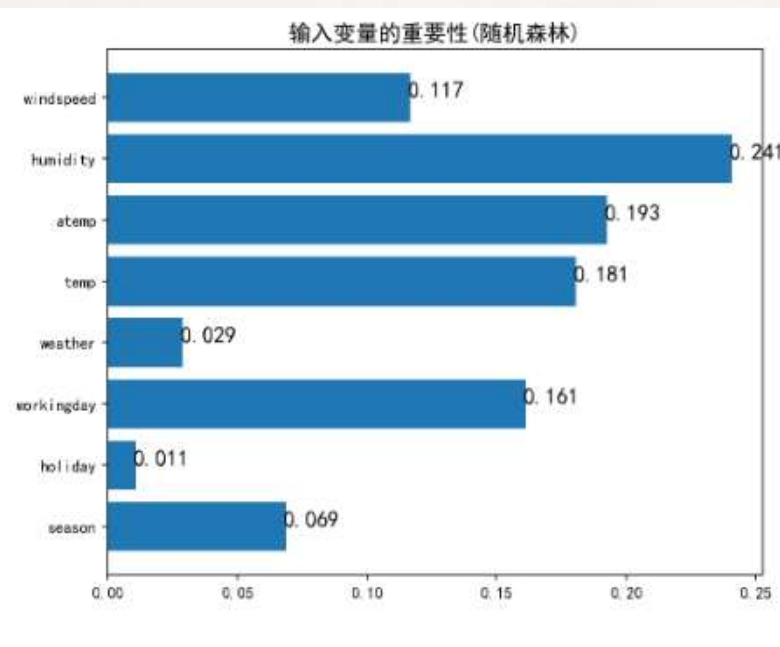
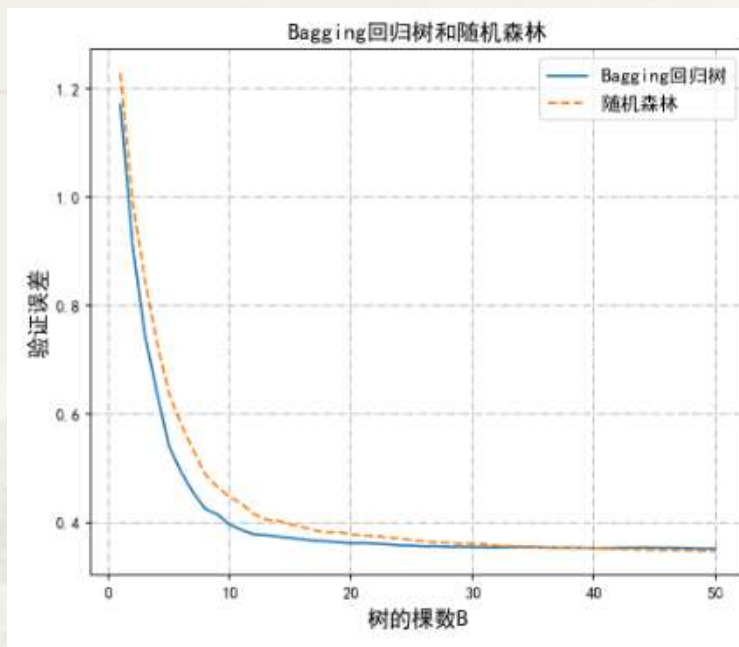
- 问:

- 常规骑行日租赁量的预测会怎样?(课后自行练习)
- 随机森林中变量重要性是如何测度的? 与之前结论不一致

通过对多个模型模型预测结果的平均, 降低预测误差

- 随机森林中的**输入变量重要性**：添加随机噪声，度量输入变量的重要性
 - 基本思路：若某输入变量对输出变量预测有重要作用，那么针对模型的OOB在该输入变量上添加随机噪声，将显著影响模型OOB的计算结果
- 步骤：
 - 对 $\hat{T}^{*(b)}$ ($b = 1, 2, \dots, B$) 计算基于OOB的测试误差，记为 $e(\hat{T}^{*(b)})$
 - 测度第 j 个输入变量对输出变量的重要性：
 - 随机打乱 $\hat{T}^{*(b)}$ 的OOB在第 j 个输入变量上的取值顺序，重新计算 $\hat{T}^{*(b)}$ 的基于OOB的误差，记为 $e^j(\hat{T}^{*(b)})$
 - 计算第 j 个输入变量添加噪声前后 $\hat{T}^{*(b)}$ 的OOB误差的变化
 - $c_{\hat{T}^{*(b)}}^j = e^j(\hat{T}^{*(b)}) - e(\hat{T}^{*(b)})$
 - 重复上述步骤 B 次，得到 B 个 $c_{\hat{T}^{*(b)}}^j$, ($b = 1, 2, \dots, B$)
- 计算均值 $\frac{1}{B} \sum_{b=1}^B c_{\hat{T}^{*(b)}}^j$ ，为第 j 个输入变量添加噪声导致的随机森林总的OOB误差变化
 - 变化值越大第 j 个输入变量越重要

- 案例二的分析：线性回归分析→K-近邻法→决策树→Bagging/随机森林



```
dtrTestErr = mean_absolute_error(y_test,modelDTC.predict(X_test))
bagTestErr = mean_absolute_error(y_test,Bag.predict(X_test))
rfTestErr = mean_absolute_error(y_test,RF.predict(X_test))
print(f'测试误差(MAE): \n    单课决策树={dtrTestErr:.3f}\n    Bagging={bagTestErr:.3f}\n    随机森林={rfTestErr:.3f}')
```

测试误差(MAE):
单课决策树=19.961
Bagging=18.928
随机森林=18.011

- 问：从集成学习角度看，误差还有进一步降低的空间吗？
 - 让各基学习器聚焦不同的局部数据并进行联合预测？
 - 再看从弱模型到强模型的向前式分步可加建模策略
 - 梯度提升算法和梯度提升树
 - 梯度提升树的XGBoost、LightGBM算法，等